Ένα αντικείμενο LocalDate αναπαριστά μια ημερομηνία που συμπεριλαμβάνει ημέρα, μήνα και έτος, χωρίς να συμπεριλαμβάνει την ώρα.

```
static void usage() {//Κώδικας 12.10
    LocalDate today = LocalDate.now();
    LocalDate dec15_71 = LocalDate.of(1971, Month.DECEMBER, 15);
    LocalDate jan1_71 = LocalDate.of(1971, 1, 15);
    LocalDate tommorow = today.plusDays(1);
    LocalDate afterAWeek = today.plusWeeks(1);
    LocalDate afterAMonth = today.plusMonths(1);
    LocalDate afterAYear = today.plusYears(1);
    LocalDate yesterday = today.minusDays(1);
    LocalDate beforeAWeek = today.minusWeeks(1);
    LocalDate beforeAMonth = today.minusMonths(1);
    LocalDate beforeAYear = today.minusYears(1);
    LocalDate afterAMonthAndAWeek = today.plusMonths(1).plusWeeks(1);
}
```

Κώδικας 12.10 *Παράδειγμα χρήσης της LocalDate*

Όπως φαίνεται στον κώδικα 12.10, η δημιουργία ενός αντικειμένου LocalDate που αναπαριστά τη σημερινή ημερομηνία γίνεται με κλήση της στατικής μεθόδου now. Για να δημιουργήσουμε μια συγκεκριμένη ημερομηνία χρησιμοποιούμε τη στατική μέθοδο of. Στον κώδικα 12.10 δημιουργούνται δύο τέτοιες ημερομηνίες, η dec15_71 και η jan1_71. Στη θέση της δεύτερης παραμέτρου της of που αναπαριστά τον μήνα, μπορεί να χρησιμοποιηθεί int ή ο απαριθμήσιμος τύπος Month. Στη συνέχεια επιδεικνύονται οι μέθοδοι plus και minus με τις οποίες μπορούμε να προσθέτουμε ή να αφαιρούμε από μια ημερομηνία. Προσέξτε πως αυτές οι μέθοδοι δεν επηρεάζουν το αντικείμενο που τις καλεί αλλά επιστρέφουν ένα νέο αντικείμενο με το ανάλογο περιεχόμενο. Η κλάση διαθέτει αναγνώστες όπως οι getYear(), getMonth(), getDayOfWeek(), getDayOfMonth(), κλπ.

12.6.2 Η κλάση LocalTime

Η κλάση LocalTime αναπαριστά την ώρα, σε ώρες, λεπτά, δευτερόλεπτα και δισεκατομυριοστά του δευτερολέπτου.

```
static void usage() {
    LocalTime currentTime = LocalTime.now();
    LocalTime t12_50 = LocalTime.of(12, 50);
    LocalTime t12_50_43 = LocalTime.of(12, 50, 43);
    LocalTime t12_50_43_625 = LocalTime.of(12, 50, 43, 625);
    LocalTime after2Hours = LocalTime.now().plusHours(2);
    LocalTime after2Minutes = LocalTime.now().plusMinutes(2);
    LocalTime after2Seconds = LocalTime.now().plusSeconds(2);
    LocalTime after1MillionNanos = LocalTime.now().plusNanos(1_000_000);
    LocalTime beforeAnHour = LocalTime.now().minusHours(1);
    LocalTime before2minutes = LocalTime.now().minusMinutes(2);
    LocalTime before3Seconds = LocalTime.now().minusSeconds(3);
    LocalTime before10Nanos = LocalTime.now().minusNanos(10);
}
```

Κώδικας 12.11 *Παράδειγμα χρήσης της LocalTime*

Όπως φαίνεται στον κώδικα 12.11, η μέθοδος now χρησιμοποιείται για τη δημιουργία αντικειμένου που αντιπροσωπεύει την ώρα του συστήματος κατά τον χρόνο εκτέλεσης της μεθόδου. Με τη μέθοδο of μπορούμε να δημιουργήσουμε αντικείμενα που αναπαριστούν συγκεκριμένες ώρες. Κατά τη δημιουργία τέτοιων αντικειμένων πρέπει υποχρεωτικά να καθορίσουμε την ώρα και τα λεπτά και προαιρετικά, τα δευτερόλεπτα και τα νανοδευτερόλεπτα. Στη συνέχεια, γίνεται επίδειξη των μεθόδων με τις οποίες προσθέτουμε ή αφαιρούμε χρόνο από ένα αντικείμενο LocalTime. Και εδώ, όπως και στην LocalDate, το αντικείμενο που καλεί τη μέθοδο

παραμένει αμετάβλητο, ενώ επιστρέφεται ένα νέο αντικείμενο. Η κλάση παρέχει κατάλληλους αναγνώστες, όπως getHour(), getMinute(), getSecond() και getNano().

Ας σημειωθεί ότι παρέχεται η κλάση LocalDateTime η οποία συνδυάζει ημερομηνία και ώρα σε ένα αντικείμενο.

12.6.3 Οι κλάσεις Period και Duration

Η κλάση Period αναπαριστά χρονικές περιόδους στη μορφή έτη, μήνες, ημέρες.

```
    static void usage() {
        Period p = Period.of(0, 3, 10),
                p1 = Period.of(3, 1, 40);
        System.out.println(Period.ofWeeks(6));
        System.out.println(Period.ofDays(60));
        System.out.println(Period.ofMonths(3));
        System.out.println(Period.ofYears(2));

        LocalDate today = LocalDate.now();
        LocalDate myBirthday = LocalDate.of(1981, Month.DECEMBER, 15);
        Period myAge = Period.between(myBirthday, today);
        System.out.println(myAge);

        Period fiveMonths = Period.ofMonths(5);
        LocalDate afterFiveMonths = today.plus(fiveMonths);
        System.out.println(afterFiveMonths);
    }
```

Κώδικας 12.12 *Παράδειγμα χρήσης της Period*

Ο κώδικας 12.12, καταρχάς επιδεικνύει κάποιες από τις δυνατότητες δημιουργίας αντικειμένων τύπου Period. Στη συνέχεια, επιδεικνύει πώς μπορούμε να υπολογίσουμε τη χρονική περίοδο μεταξύ δύο αντικειμένων LocalDate με τη μέθοδο between και τέλος πώς μπορούμε να προσθέσουμε μια περίοδο σε μία ημερομηνία. Η έξοδος του κώδικα 12.12 είναι η εξής:

```
P42D
P60D
P3M
P2Y
P39Y9M15D
2022-02-28
```

Η κλάση Duration μοντελοποιεί μια χρονική διάρκεια σε δευτερόλεπτα ή/και νανοδευτερόλεπτα.

```
    static void usage() {
        Duration classDuration = Duration.ofMinutes(330);
        Duration breakTime=Duration.ofMinutes(4*10);
        Duration study=classDuration.minus(breakTime);
        System.out.println(study);

        LocalTime startLessons=LocalTime.of(8,0);
        LocalTime endLessons=startLessons.plus(classDuration);
        System.out.println(Duration.between(endLessons, startLessons));

System.out.println(classDuration.toHours()+":"+classDuration.toMinutesPart());
    }
```

Κώδικας 12.13 *Παραδείγματα χρήσης της Duration*

Στον κώδικα 12.13 καταρχάς δημιουργούμε τη χρονική διάρκεια classDuration 330 λεπτών και την breakTime διάρκειας 40 λεπτών. Στη συνέχεια υπολογίζουμε και τυπώνουμε τη διάρκεια του μαθήματος. Αμέσως μετά

καθορίζουμε την ώρα έναρξης του μαθήματος και υπολογίζουμε την ώρα λήξης του. Τέλος, εμφανίζουμε την classDuration στη μορφή hours:minutes.
Η έξοδος του κώδικα 12.13 έχει ως εξής:

```
PT4H50M
PT-5H-30M
5:30
```

12.6.4 Διαμόρφωση

Συχνά προκύπτει η ανάγκη διαμόρφωσης των αντικειμένων χρόνου. Η τυπική διαμόρφωση των ημερομηνιών επιτυγχάνεται με χρήση της κλάσης DateTimeFormatter.

```
    static void formatDate() {
        DateTimeFormatter shortFormat =
DateTimeFormatter.ofLocalizedDate(FormatStyle.SHORT);
        DateTimeFormatter mediumFormat =
DateTimeFormatter.ofLocalizedDate(FormatStyle.MEDIUM);
        DateTimeFormatter longFormat =
DateTimeFormatter.ofLocalizedDate(FormatStyle.LONG);
        DateTimeFormatter fullFormat =
DateTimeFormatter.ofLocalizedDate(FormatStyle.FULL);
        LocalDate d = LocalDate.of(2021, 9,30);
        System.out.println(shortFormat.format(d));
        System.out.println(d.format(mediumFormat));
        System.out.println(d.format(longFormat));
        System.out.println(d.format(fullFormat));
    }
```

Κώδικας 12.14 *Τυπική διαμόρφωση ημερομηνίας*

Στον κώδικα 12.14 έχουμε δημιουργήσει τέσσερις διαμορφωτές, τους shortFormat, mediumFormat, longFormat και fullFormat. Στη συνέχεια εμφανίζουμε τη σημερινή ημερομηνία αξιοποιώντας τη μέθοδο format της κλάσης LocalTime ή εναλλακτικά, τη μέθοδο format της κλάσης DateTimeFormatter. Η έξοδος του κώδικα έχει ως εξής:

```
30/9/21
30 Σεπ 2021
30 Σεπτεμβρίου 2021
Πέμπτη, 30 Σεπτεμβρίου 2021
```

Η τυπική διαμόρφωση της ώρας επιτυγχάνεται επίσης με χρήση της DateTimeFormatter.

```
    static void formatTime() {//Κώδικας 12.15
        DateTimeFormatter shortFormat =
DateTimeFormatter.ofLocalizedTime(FormatStyle.SHORT);
        DateTimeFormatter mediumFormat =
DateTimeFormatter.ofLocalizedTime(FormatStyle.MEDIUM);
        LocalTime t = LocalTime.of(12, 50, 43);
        System.out.println(shortFormat.format(t));
        System.out.println(t.format(mediumFormat));
    }
```

Κώδικας 12.15 *Τυπική διαμόρφωση ώρας*

Ο κώδικας 12.15 παρέχει παραδείγματα τυπικής διαμόρφωσης της ώρας. Προσοχή, οι διαμορφωτές FormatStyle.LONG και FormatStyle.FULL δεν ισχύουν για τη διαμόρφωση της ώρας. Η έξοδος του κώδικα 12.15 έχει ως εξής:

```
12:50 μ.μ.
12:50:43 μ.μ.
```

Πέραν της τυπικής διαμόρφωσης, είναι δυνατόν να καθορίσουμε τη διαμόρφωση με βάση κάποιο μοτίβο με τη βοήθεια της συνάρτησης ofPattern.

```
static void customFormat() {
    LocalDate d = LocalDate.now();
    DateTimeFormatter f1 = DateTimeFormatter.ofPattern("MMMM, dd, yyyy");
    DateTimeFormatter f2 = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    DateTimeFormatter f3 = DateTimeFormatter.ofPattern("dd-MMMM-yy");
    System.out.println(d.format(f2));
    System.out.println(d.format(f3));

    DateTimeFormatter dtf = DateTimeFormatter.ofPattern("hh:mm:ss.nn a");
    LocalTime t = LocalTime.of(15,20,30,450);
    System.out.println(t.format(dtf));
    dtf = DateTimeFormatter.ofPattern("hh:mm a");
    System.out.println(t.format(dtf));
}
```

Κώδικας 12.16 *Διαμόρφωση ημερομηνίας και ώρας με βάση μοτίβο*

Ο κώδικας 12.16 παρουσιάζει παραδείγματα διαμόρφωσης ημερομηνιών και ωρών. Εύκολα μπορεί να καταλάβει κανείς από τον κώδικα πως ο χαρακτήρας y αντιστοιχεί σε ψηφίο έτους, ο M σε ψηφίο μήνα, ο d σε ψηφίο ημέρας, ο h σε ψηφίο ώρας, ο m σε ψηφίο λεπτού, ο s σε ψηφίο δευτερολέπτου και ο n σε ψηφίο νανοδευτερολέπτου. Τέλος, ο χαρακτήρας a έχει ως αποτέλεσμα η ώρα να συνοδεύεται από τη σύντμηση π.μ. ή μ.μ., ανάλογα με την τιμή της ώρας.
Η έξοδος του κώδικα 12.16 έχει ως εξής:

```
30/09/2021
30-Σεπτεμβρίου-21
03:20:30.450 μ.μ.
03:20 μ.μ.
```

Η κλάση DateTimeFormatter μπορεί να αξιοποιηθεί και κατά τη δημιουργία αντικειμένων ημερομηνίας και ώρας.

```
static void parsing() { //Κώδικας 12.17
    DateTimeFormatter fd = DateTimeFormatter.ofPattern("dd/MM/yyyy");
    DateTimeFormatter ft = DateTimeFormatter.ofPattern("hh:mm:ss");
    LocalDate d = LocalDate.parse("15/01/1987", fd);
    LocalTime t = LocalTime.parse("12:35:31", ft);
    System.out.println(d);
    System.out.println(t);
}
```

Κώδικας 12.17 *Δημιουργία ημερομηνίας και ώρας με βάση μοτίβο*

Στον κώδικα 12.17 έχουμε δημιουργήσει δύο διαμορφωτές, τον fd και τον ft, για ημερομηνίες και ώρες, αντίστοιχα. Στη συνέχεια, αξιοποιώντας τους διαμορφωτές, δημιουργούμε αντικείμενα LocalDate και LocalTime περνώντας στην parse τις τιμές ημερομηνίας ή ώρας σαν συμβολοσειρές που ακολουθούν το μοτίβο του αντίστοιχου διαμορφωτή.

12.7 Λυμένες Ασκήσεις

12.7.1 Η κλάση τράπουλα

Αντικαταστήστε τον πίνακα deck της κλάσης Deck (άσκηση 11.9.7) με κατάλληλο ArrayList.

Λύση

```
import java.util.ArrayList;
import java.util.Collections;

public class Deck {

    private final ArrayList<Card> deck;

    public Deck() {
        deck = new ArrayList<>(52);
        for (CardRank r : CardRank.values()) {
            for (Suit s : Suit.values()) {
                deck.add(new Card(r, s));
            }
        }
    }

    public void shuffle() {
        Collections.shuffle(deck);
    }

    public boolean empty() {
        return deck.isEmpty();
    }

    public Card pickCard() {
        return deck.remove(0);
    }

    @Override
    public String toString() {
        return deck.toString();
    }

    public static void main(String[] args) {
        Deck d1 = new Deck();
        Deck d2 = new Deck();
         for (int i = 0; i < 24; i++) {
            d1.pickCard();
        }
        System.out.println(d1);
        System.out.println("");
        d2.shuffle();
        System.out.println(d2);
    }
}
```

Κώδικας 12.18 *Η κλάση Deck με ArrayList*

Όπως φαίνεται στον κώδικα 12.18, η αντικατάσταση του πίνακα με ArrayList, απλοποιεί σημαντικά τον κώδικα. Η ιδιωτική μεταβλητή cardsInDeck δεν είναι πλέον απαραίτητη. Για το ανακάτεμα της τράπουλας δεν είναι αναγκαίο να αναπτύξουμε δικό μας αλγόριθμο, καθώς είναι διαθέσιμη κατάλληλη συνάρτηση από την κλάση Collections. Επίσης, η συνάρτηση toString απλοποιείται σημαντικά.

12.7.2 Δεύτερη έκδοση της κλάσης Person

Να υλοποιηθεί νέα έκδοση της κλάσης Person. Στη νέα έκδοση, κάθε αντικείμενο Person διαθέτει αριθμό ταυτότητας, μικρό όνομα, επώνυμο και ημερομηνία γέννησης. Ο αριθμός ταυτότητας είναι μια συμβολοσειρά που προσδιορίζει μονοσήμαντα ένα πρόσωπο, δηλαδή θεωρούμε πως δύο στιγμιότυπα με τον ίδιο αριθμό ταυτότητας αφορούν το ίδιο πρόσωπο. Επομένως, μετά τη δημιουργία ενός αντικειμένου Person, ο αριθμός ταυτότητας δεν μπορεί να μεταβληθεί. Η κλάση ορίζει όλες τις αναγκαίες μεθόδους και επιπλέον μια μέθοδο που επιστρέφει την ηλικία του προσώπου σε ακέραιο αριθμό ετών. Προσθέστε μέθοδο main στην οποία να δημιουργήσετε δύο αντικείμενα Person και να εμφανίσετε τις ηλικίες τους.

Λύση
Παραθέτουμε πρώτα τον κώδικα της κλάσης Person και μετά επεξηγήσεις.

```
import java.time.LocalDate;
import java.time.Month;
import java.time.Period;

public class Person {

    private final String id;
    private String fName;
    private String sName;
    private LocalDate birthday;

    public Person(String id, String fName, String sName, LocalDate birthday) {
        this.id = id;
        this.fName = fName;
        this.sName = sName;
        this.birthday = birthday;
    }

    public LocalDate getBirthday() {
        return birthday;
    }

    public void setBirthday(LocalDate birthday) {
        this.birthday = birthday;
    }

    public String getfName() {
        return fName;
    }

    public void setfName(String fName) {
        this.fName = fName;
    }

    public String getsName() {
        return sName;
    }

    public void setsName(String sName) {
        this.sName = sName;
    }

    public int getAge() {
        LocalDate today = LocalDate.now();
        return Period.between(birthday, today).getYears();
    }

    public String getId() {
        return id;
```

```
    }

    @Override
    public int hashCode() {
        int hash = 5;
        return 17 * hash + this.id.hashCode();
    }

    @Override
    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (obj == null) {
            return false;
        }
        if (getClass() != obj.getClass()) {
            return false;
        }
        final Person other = (Person) obj;
        return this.id.equals(other.id);
    }

    @Override
    public String toString() {
        return fName + " " + sName + ", id=" + id;
    }

    public static void main(String[] args) {
        Person jim = new Person("A89132", "Jim", "Black", LocalDate.of(1971,
Month.MARCH, 5));
        Person john = new Person("A89132", "John", "Black", LocalDate.of(1972,
Month.APRIL, 5));
        System.out.println(jim + " is " + jim.getAge() + " years old ");
        System.out.println(john + " is " + john.getAge() + " years old ");
    }
}
```

Κώδικας 12.19 *Η δεύτερη έκδοση της κλάσης Person*

Ο δημιουργός της κλάσης Person, οι ρυθμιστές και οι αναγνώστες συνιστούν τυπικές υλοποιήσεις. Επειδή ο αριθμός ταυτότητας ταυτοποιεί ένα πρόσωπο είναι ιδιωτικός και final και επομένως δεν υπάρχει ρυθμιστής για τον αριθμό ταυτότητας, δηλαδή δεν μπορεί να μεταβληθεί μετά τη δημιουργία αντικειμένου Person.

Από τη στιγμή που θεωρούμε πως δύο αντικείμενα Person με τον ίδιο αριθμό ταυτότητας αφορούν το ίδιο πρόσωπο, οι συναρτήσεις equals και hashCode μπορούν να βασιστούν αποκλειστικά στο id.

Η συνάρτηση getAge βασίζεται στο παράδειγμα χρήσης της Period (κώδικας 12.12) με τη διαφορά ότι εκφράζει την ηλικία του προσώπου μόνον σε έτη.

Στην main δημιουργούμε δύο πρόσωπα και εμφανίζουμε τις ηλικίες τους σύμφωνα με την απαίτηση της εκφώνησης.

12.7.3 Η κλάση MyHashMap

Να αναπτυχθεί η κλάση MyHashMap που συσχετίζει ένα αντικείμενο Person (κώδικας 12.19) με το id του. Η κλάση να περιλαμβάνει τέσσερις μόνο μεθόδους και πιο συγκεκριμένα τις:

```
public Person put(Person person)
```

Εισάγει στο αντικείμενο MyHashMap, ένα αντικείμενο Person. Αν αντικείμενο Person με το ίδιο id δεν υπάρχει ήδη στο MyHashMap, η put το εισάγει και επιστρέφει null. Αν αντικείμενο Person με το ίδιο id βρίσκεται ήδη στο αντικείμενο MyHashMap, η put το επιστρέφει και ενημερώνει το MyHashMap με τη νέα τιμή.

`Person get(String id)`
Επιστρέφει το αντικείμενο Person με αριθμό ταυτότητας id ή null αν δεν βρεθεί το id.

`Person remove(String id)`
Αφαιρεί και επιστρέφει το αντικείμενο Person με αριθμό ταυτότητας id. Αν δεν βρεθεί αυτό το id επιστρέφει null.

`toString()`.
Επιστρέφει την String αναπαράσταση των αντικειμένων Person που περιέχονται στο MyHashMap, ένα ανα γραμμή.
Προσθέστε συνάρτηση main στην οποία

1. Δημιουργήστε ένα αντικείμενο MyHashMap
2. Εισάγετε 6 αντικείμενα Person
3. Διαβάστε με την get, από το αντικείμενο MyHashMap, ένα-ένα τα πρόσωπα που έχετε εισάγει και τυπώστε τα, ένα ανά γραμμή.
4. Μεταβάλλετε το μικρό όνομα ενός από τα 6 αντικείμενα Person και ενημερώστε το αντικείμενο MyHashMap. Διαβάστε από το MyHashMap το πρόσωπο του οποίου μεταβλήθηκε το όνομα και τυπώστε το ώστε να ελέγξετε πως η ενημέρωση έγινε σωστά.
5. Διαγράψτε ένα πρόσωπο από το αντικείμενο MyHashMap και τυπώστε το αντικείμενο ώστε να ελέγξετε πως η μέθοδος toString() δουλεύει σύμφωνα με την εκφώνηση.
6. Η κλάση δεν επιτρέπει την καταχώρηση αντικειμένων με null τιμή. Η άσκηση να λυθεί με τη λογική που παρουσιάζει το σχήμα 12.3, χωρίς να χρησιμοποιηθεί η κλάση HashMap.

Λύση

```
import java.time.LocalDate;
import java.time.Month;
import java.util.ArrayList;

public class MyHashMap {

    private static final int SIZE = 3;
    private final ArrayList<Person>[] store = new ArrayList[SIZE];

    public Person put(Person person) {
        if (person == null) {
            return null;
        }
        String id = person.getId();
        int hashCode = id.hashCode();
        int idx = hashCode % store.length;
        if (store[idx] == null) {
            store[idx] = new ArrayList<>();
        }
        int idLocation = -1;
        for (int i = 0; i < store[idx].size(); i++) {
            if (store[idx].get(i).getId().equals(id)) {
                idLocation = i;
                break;
            }
        }

        Person oldValue;
        if (idLocation == -1) {
            store[idx].add(person);
            return null;
        } else {
```

```
            oldValue = store[idx].get(idLocation);
            store[idx].set(idLocation, person);
        }
        return oldValue;
    }

    Person get(String id) {
        int hashCode = id.hashCode();
        int idx = hashCode % store.length;
        if (store[idx] == null) {
            return null;
        }
        int idLocation = -1;
        for (int i = 0; i < store[idx].size(); i++) {
            if (store[idx].get(i).getId().equals(id)) {
                idLocation = i;
                break;
            }
        }
        if (idLocation == -1) {
            return null;
        }
        return store[idx].get(idLocation);
    }

    Person remove(String id) {
        int hashCode = id.hashCode();
        int idx = hashCode % store.length;
        if (store[idx] == null) {
            return null;
        }
        int idLocation = -1;
        for (int i = 0; i < store[idx].size(); i++) {
            if (store[idx].get(i).getId().equals(id)) {
                idLocation = i;
                break;
            }
        }
        if (idLocation == -1) {
            return null;
        }
        Person rVal = store[idx].get(idLocation);
        store[idx].remove(idLocation);
        return rVal;
    }

    @Override
    public String toString() {
        StringBuilder rVal = new StringBuilder("");
        for (ArrayList<Person> cStore : store) {
            if (cStore != null) {
                for (int j = 0; j < cStore.size(); j++) {
                    rVal.append(cStore.get(j)).append("\n");
                }
            }
        }
        return rVal.toString();
    }

    public static void main(String[] args) {
```

```
        Person p1 = new Person("A1", "Jim", "Black", LocalDate.of(1961,
Month.MARCH, 1));
        Person p2 = new Person("A2", "Jack", "Blue", LocalDate.of(1967,
Month.MARCH, 1));
        Person p3 = new Person("A3", "Jim", "Green", LocalDate.of(1971,
Month.MARCH, 1));
        Person p4 = new Person("A4", "Mary", "White", LocalDate.of(1961,
Month.MARCH, 1));
        Person p5 = new Person("A5", "Ann", "Red", LocalDate.of(1967,
Month.MARCH, 1));
        Person p6 = new Person("A6", "Kelly", "Pink", LocalDate.of(1971,
Month.MARCH, 1));
        MyHashMap map = new MyHashMap();
        map.put(p1);
        map.put(p2);
        map.put(p3);
        map.put(p4);
        map.put(p5);
        map.put(p6);

        System.out.println(map.get(p1.getId()));
        System.out.println(map.get(p2.getId()));
        System.out.println(map.get(p3.getId()));
        System.out.println(map.get(p4.getId()));
        System.out.println(map.get(p5.getId()));
        System.out.println(map.get(p6.getId()));

        p1.setfName("Jameson");
        map.put(p1);
        System.out.println("after fName replacement");
        System.out.println(map.get(p1.getId()));

        System.out.println(map.remove(p1.getId()));
        System.out.println("after remove One");
        System.out.println(map.get(p1.getId()));
        System.out.println("print map");
        System.out.println(map);
    }
}
```

Κώδικας 12.20 *Η κλάση MyHashMap*

Το πιο κρίσιμο σημείο για τη λύση αυτής της άσκησης είναι η επιλογή της κατάλληλης δομής. Στη λύση που προτείνεται στον κώδικα 12.20, έχει επιλεγεί ένας πίνακας από ArrayList. Κάθε ένα ArrayList περιέχει αντικείμενα Person. Η put λαμβάνει ως παράμετρο ένα Person. Στην τυπική περίπτωση, η put θα λάμβανε δύο παραμέτρους, ένα String που θα αντιπροσώπευε το id και θα ήταν το κλειδί της απεικόνισης και ένα Person που θα συσχετιζόταν με το id. Στη συγκεκριμένη εφαρμογή όμως γνωρίζουμε πως κάθε Person συσχετίζεται αποκλειστικά με το id του. Επομένως, το κλειδί της απεικόνισης είναι το id το οποίο μπορούμε να πάρουμε από την παράμετρο Person της put.

Στην περίπτωση που η put κληθεί με παράμετρο null απλώς επιστρέφει. Στη συνέχεια παράγει την τιμή κατακερματισμού και εντοπίζει τη θέση στον πίνακα store υπολογίζοντας το ακέραιο υπόλοιπο της τιμής κατακερματισμού με το μέγεθος του store. Αν στη θέση που αντιστοιχεί το id δεν υπάρχει λίστα, τότε αυτή δημιουργείται. Στη συνέχεια εισάγεται το αντικείμενο Person εφόσον αυτό δεν υπάρχει ήδη στην αντίστοιχη λίστα.

Η get λαμβάνει ως παράμετρο ένα id. Υπολογίζει σε ποια θέση του πίνακα αντιστοιχεί αυτό το id με παρόμοιο τρόπο με την put. Αν στη θέση εκείνη δεν υπάρχει λίστα, είναι προφανές πως το κλειδί και το αντίστοιχο πρόσωπο δεν έχουν καταχωρηθεί, οπότε επιστρέφει null. Αν βρεθεί λίστα, ανιχνεύεται σειριακά για το id. Αν βρεθεί επιστρέφεται το αντικείμενο Person, διαφορετικά null.

Αντίστοιχα λειτουργεί η remove, με τη διαφορά πως αν βρεθεί το αντικείμενο, διαγράφεται από την αντίστοιχη λίστα.

Η toString χρησιμοποιεί StringBuilder για να διαμορφώσει την τιμή επιστροφής της. Αφού τη διαμορφώσει την επιστρέφει σε μορφή String. Προσθέτει στην rVal, κάθε Person, κάθε λίστας του store. Μετά από κάθε Person προσθέτει τον χαρακτήρα '\n' ώστε να αλλάζει γραμμή σύμφωνα με τις απαιτήσεις της εκφώνησης.

Η main ακολουθεί απλώς τις οδηγίες της εκφώνησης. Προσέξτε το σημείο κατά το οποίο μεταβάλλουμε το μικρό όνομα ενός προσώπου, δηλαδή τον κώδικα

```
p1.setfName("Jameson");
map.put(p1);
System.out.println("after fName replacement");
System.out.println(map.get(p1.getId()));
```

Τι θα συμβεί αν διαγράψουμε την πρόταση map.put(p1). Κάντε μια υπόθεση. Προχωρήστε στη διαγραφή και τρέξτε πάλι την κλάση. Μαντέψατε σωστά; Αν όχι, εξηγήστε αυτό το αποτέλεσμα.

Τέλος, ας σημειωθεί πως στον κώδικα 12.20 ο πίνακας store δημιουργήθηκε με μέγεθος μόλις 3 για λόγους διευκόλυνσης του ελέγχου του κώδικα. Με αυτό το μέγεθος ακόμη και λίγα αντικείμενα Person αντιστοιχίζονται εύκολα στην ίδια θέση του πίνακα και επομένως προστίθενται περισσότερα από 1 αντικείμενα στην αντίστοιχη λίστα.

12.7.4 Η κλάση MyPriorityQueue

Σε ένα νοσοκομείο, οι ασθενείς ταξινομούνται σε τρεις σειρές προτεραιότητας, ασθενείς με χαμηλή, με μεσαία και με υψηλή προτεραιότητα. Οι ασθενείς εξυπηρετούνται ανάλογα με την προτεραιότητά τους. Μεταξύ ασθενών με την ίδια προτεραιότητα, εξυπηρετούνται πρώτα αυτοί που εισάγονται πρώτοι. Να υλοποιηθεί η κλάση MyPriorityQueue που υποστηρίζει μια ουρά προτεραιότητας κατάλληλη για τις ανάγκες εξυπηρέτησης των ασθενών του νοσοκομείου. Υλοποιήστε δύο βασικές μεθόδους, την add με την οποία εισάγεται ένας ασθενής στο σύστημα και την get η οποία επιστρέφει έναν ασθενή σύμφωνα με την κατάλληλη σειρά και παράλληλα τον διαγράφει από την ουρά. Για ασθενείς, χρησιμοποιήστε αντικείμενα της κλάσης Person (κώδικας 12.19). Προσθέστε main στην οποία δημιουργήστε ένα αντικείμενο MyPriorityQueue, εισάγεται έξι πρόσωπα, δύο με χαμηλή προτεραιότητα, δύο με μεσαία και δύο με υψηλή. Εξάγετε από την ουρά όλα τα πρόσωπα. Εμφανίστε τα και ελέγξτε ότι εξάγονται με τη σωστή σειρά.

Λύση

```
import java.time.LocalDate;
import java.time.Month;
import java.util.LinkedList;

public class MyPriorityQueue {

    private final LinkedList<Person>[] store;

    public MyPriorityQueue() {
        store = new LinkedList[3];
        store[Priority.LOW.ordinal()] = new LinkedList<>();
        store[Priority.NORMAL.ordinal()] = new LinkedList<>();
        store[Priority.HIGH.ordinal()] = new LinkedList<>();
    }

    public void add(Priority priority, Person person) {
        if (store[priority.ordinal()].contains(person)) {
            return;
        }
        store[priority.ordinal()].addLast(person);
    }

    public Person get() {
        if (!store[Priority.HIGH.ordinal()].isEmpty()) {
```

```
            return store[Priority.HIGH.ordinal()].removeFirst();
        }
        if (!store[Priority.NORMAL.ordinal()].isEmpty()) {
            return store[Priority.NORMAL.ordinal()].removeFirst();
        }
        if (!store[Priority.LOW.ordinal()].isEmpty()) {
            return store[Priority.LOW.ordinal()].removeFirst();
        }
        return null;
    }

    public static void main(String[] args) {
        Person p1 = new Person("A1", "Jim", "Black", LocalDate.of(1961,
Month.MARCH, 1));
        Person p2 = new Person("A2", "Jack", "Blue", LocalDate.of(1967,
Month.MARCH, 1));
        Person p3 = new Person("A3", "Jim", "Green", LocalDate.of(1971,
Month.MARCH, 1));
        Person p4 = new Person("A4", "Mary", "White", LocalDate.of(1961,
Month.MARCH, 1));
        Person p5 = new Person("A5", "Ann", "Red", LocalDate.of(1967,
Month.MARCH, 1));
        Person p6 = new Person("A6", "Kelly", "Pink", LocalDate.of(1971,
Month.MARCH, 1));
        MyPriorityQueue pQueue = new MyPriorityQueue();
        pQueue.add(Priority.LOW, p1);
        pQueue.add(Priority.LOW, p2);
        pQueue.add(Priority.NORMAL, p3);
        pQueue.add(Priority.NORMAL, p4);
        pQueue.add(Priority.HIGH, p5);
        pQueue.add(Priority.HIGH, p6);
        Person p;
        while ((p = pQueue.get()) != null) {
            System.out.println(p);
        }
    }

}
```

Κώδικας 12.21 *Η κλάση MyPriorityQueue*

Όπως φαίνεται στον κώδικα 12.21, η κύρια δομή της κλάσης MyPriorityQueue είναι ένας πίνακας από LinkedList. Η κάθε λίστα του πίνακα φιλοξενεί αντικείμενα τύπου Person. Η Priority είναι απαριθμήσιμος τύπος με τρείς τιμές, LOW, NORMAL και HIGH.

Στον δημιουργό της MyPriorityQueue, δημιουργείται καταρχάς ο πίνακας store με σταθερό μέγεθος 3, δηλαδή ένα στοιχείο για κάθε δυνατή προτεραιότητα. Για κάθε στοιχείο του πίνακα δημιουργείται μια LinkedList την οποία χειριζόμαστε σαν δομή FIFO.

Η add λαμβάνει δύο παραμέτρους, την προτεραιότητα και τον ασθενή. Από την προτεραιότητα εντοπίζει την επιμέρους ουρά προτεραιότητας στο τέλος της οποίας προσθέτει τον ασθενή εφόσον δεν υπάρχει ήδη σε αυτήν την ουρά.

Η get ελέγχει τις επιμέρους ουρές προτεραιότητας. Αν βρει ουρά υψηλής προτεραιότητας, επιστρέφει τον ασθενή στην αρχή της ουράς και ταυτοχρόνως τον διαγράφει από την ουρά. Αν η ουρά υψηλής προτεραιότητας είναι άδεια, προχωράει και ελέγχει κατά σειρά την ουρά μεσαίας προτεραιότητας και την ουρά χαμηλής προτεραιότητας. Αν όλες οι ουρές είναι άδειες, επιστρέφει null. Επομένως, ένας τρόπος για να ελέγξουμε αν η MyPriorityQueue είναι άδεια είναι αν η get μας επιστρέψει null. Αυτόν ακριβως τον τρόπο αξιοποιούμε στην main καθώς αδειάζουμε το αντικείμενο pQueue.

12.8 Ασκήσεις προς Λύση